Cypher is Turing-Complete

A Formal Proof via 2-Counter Machine Simulation

Pierre Halftermeyer · Neo4j

March 2026

Abstract. We prove that Cypher 25, the graph query language of Neo4j, is Turing-complete. The proof proceeds by showing that a single `RETURN` statement using `reduce()`, `CASE` expressions, and list comprehensions can simulate any 2-counter machine (Minsky machine). Since 2-counter machines are known to be Turing-complete, the result follows by reduction. We address the bounded-step objection inherent to `reduce()` via two complementary resolutions, and present a second, graph-native simulation using quantified path patterns (QPP) with `allReduce`, where the machine *is* the graph and computation *is* path finding. All three constructions are verified on a live Neo4j Aura instance.

§1 The Claim

Cypher 25, restricted to a single `RETURN` statement with `reduce()`, list comprehensions, and `CASE` expressions, **is Turing-complete**.

§1.1 Proof Strategy

We prove Turing-completeness by *reduction*:

$$\text{Turing Machine} \xrightarrow{\text{Minsky 1967}} \text{2-Counter Machine} \xrightarrow{\text{this proof}} \text{Cypher } \texttt{reduce()}$$

- 2-counter machines are known to be Turing-complete [1].
- We show Cypher can simulate *any* 2-counter machine.
- Therefore Cypher is Turing-complete.

§2 2-Counter Machines

§2.1 Definition

A **2-counter machine** $M = (Q,\ q_0,\ q_{\text{halt}},\ \delta)$ consists of:

Component	Meaning
Q	Finite set of states
$q_0 \in Q$	Initial state
$q_{\text{halt}} \in Q$	Halting state
$\delta : Q \to$ Instructions	Transition function

Each instruction is one of:

- $\texttt{INC}(c, q')$ — increment counter $c \in \{A, B\}$, go to state q'
- $\texttt{JZDEC}(c, q_z, q_p)$ — if $c = 0$ go to q_z, else decrement c and go to q_p

Fact (Minsky 1967)

For any Turing machine T, there exists a 2-counter machine M that simulates T. Two-counter machines are the *simplest* known Turing-complete model: only two unbounded integers, only two instruction types, and a finite-state controller. If Cypher can simulate this, it can simulate *anything*.

§2.2 Example: Program P_1

The following 4-state program increments A, tests B, loops once to increment B, then halts. It computes $f(n) = 2n$ on input $B = n$, leaving the result in A.

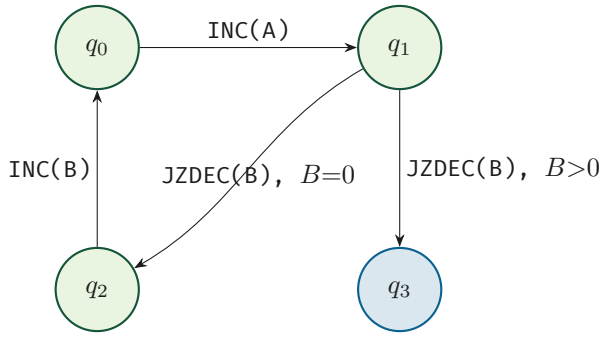

Figure 1 — Program P_1: a 4-state 2-counter machine. We use this program as the running example throughout the paper.

§2.3 Expected Execution Trace

Starting from $(q_0,\ A{=}0,\ B{=}0)$:

Step	Instruction	State	A	B
0	INC(A, q_1)	q_0	$0 \to 1$	0
1	JZDEC(B, q_2, q_3); $B{=}0$	q_1	1	0
2	INC(B, q_0)	q_2	1	$0 \to 1$
3	INC(A, q_1)	q_0	$1 \to 2$	1
4	JZDEC(B, q_2, q_3); $B{>}0$	q_1	2	$1 \to 0$
5	HALT	q_3	2	0

Expected result: $\{state : -1,\ A : 2,\ B : 0\}$

§3 Approach 1: Pure `reduce()`

This approach encodes the entire simulation as a single `RETURN` statement — no graph writes, no side effects.

§3.1 Encoding

The transition function δ becomes a Cypher **list of maps**:

```
LET program = [
  {state: 0, op: 'INC',   counter: 'A', next: 1},
  {state: 1, op: 'JZDEC', counter: 'B', q_zero: 2, q_pos: 3},
  {state: 2, op: 'INC',   counter: 'B', next: 0},
  {state: 3, op: 'HALT',  counter: '',  next: 3}
]
```

This is a *finite, static* list — no graph writes needed. `program[i]` gives $\delta(q_i)$ in $O(1)$.

The full machine configuration $(q,\ c_A,\ c_B)$ is a single Cypher **map**: `{state: 0, A: 0, B: 0}`. Convention: `state = -1` means *halted*.

§3.2 Simulation

Each iteration of `reduce()` performs **one machine step**:

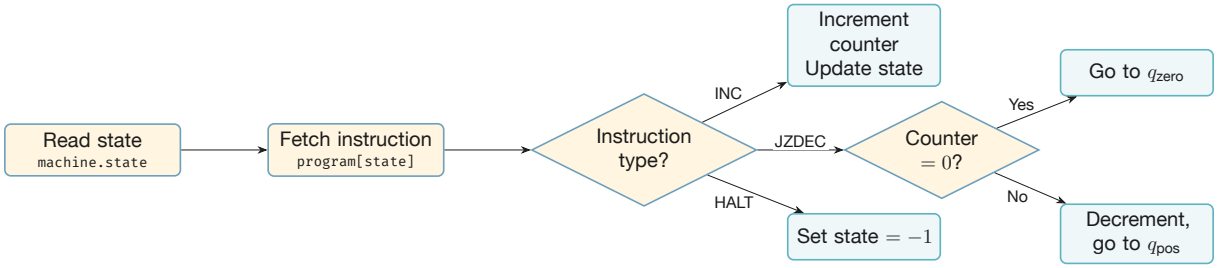

Figure 2 — Execution flow of one `reduce()` iteration.

```
CYPHER 25
LET program = [ /* ... as above ... */ ]
LET max_steps = 1000000  // arbitrary bound (see §4)

LET result = reduce(
  machine = {state: 0, A: 0, B: 0},
  step IN range(1, max_steps) |
  CASE WHEN machine.state = -1
    THEN machine  // halted: identity
  ELSE
    head([instr IN [program[machine.state]] |
      CASE instr.op
        WHEN 'INC' THEN
          CASE instr.counter
            WHEN 'A' THEN
              {state: instr.next,
               A: machine.A + 1, B: machine.B}
            WHEN 'B' THEN
              {state: instr.next,
               A: machine.A, B: machine.B + 1}
          END
        WHEN 'JZDEC' THEN
          CASE instr.counter
            WHEN 'A' THEN
              CASE WHEN machine.A = 0
                THEN {state: instr.q_zero,
                      A: 0, B: machine.B}
                ELSE {state: instr.q_pos,
                      A: machine.A - 1, B: machine.B}
              END
            WHEN 'B' THEN
              CASE WHEN machine.B = 0
                THEN {state: instr.q_zero,
                      A: machine.A, B: 0}
                ELSE {state: instr.q_pos,
                      A: machine.A, B: machine.B - 1}
              END
          END
        WHEN 'HALT' THEN
          {state: -1, A: machine.A, B: machine.B}
      END
    ])
```

```
  END
)
RETURN result
```

§3.3 Key Idiom: Let-Binding in `reduce()`

`LET` is *not available* inside `reduce()`. The workaround uses a single-element list comprehension:

```
// Let-binding idiom:
head([instr IN [program[machine.state]] |
  // 'instr' is now bound -- use it freely
  CASE instr.op WHEN 'INC' THEN ... END
])
```

This wraps the expression in a single-element list `[expr]`, the comprehension *binds* the variable name, and `head()` unwraps the result.

§3.4 Correctness

After k iterations of `reduce()`, the value of `machine` equals the configuration of machine M after k steps.

Base case ($k = 0$):

$$\texttt{machine} = \{state : 0,\ A : 0,\ B : 0\} = \text{initial configuration of } M \quad \checkmark$$

Inductive step. Assume after k iterations: $\texttt{machine} = \{state : q_k,\ A : a_k,\ B : b_k\}$ matches M's configuration.
At iteration $k{+}1$, the `reduce()` body:

1. Looks up `program[machine.state]` $\rightarrow$ retrieves instruction $\delta(q_k)$.
2. Executes via `CASE`:

Instruction	Cypher returns	Matches?
$\texttt{INC}(A, q')$	$\{state : q',\ A : a_k + 1,\ B : b_k\}$	✓
$\texttt{JZDEC}(A, q_z, q_p),\ a_k = 0$	$\{state : q_z,\ A : 0,\ B : b_k\}$	✓
$\texttt{JZDEC}(A, q_z, q_p),\ a_k > 0$	$\{state : q_p,\ A : a_k - 1,\ B : b_k\}$	✓
`HALT`	$\{state : -1, \ldots\}$ — identity thereafter	✓

Symmetric cases for counter B. Therefore `machine` after $k{+}1$ iterations matches M's configuration after $k{+}1$ steps. □

§4 The Bounded-Step Objection

The `reduce()` proof uses `range(1, max_steps)` — a *finite* bound. Strict Turing-completeness requires *unbounded* computation. We offer two complementary resolutions.

§4.1 Resolution 1: `IN TRANSACTIONS`

Cypher 25 provides iteration via `IN TRANSACTIONS` with `ON ERROR BREAK`. The machine state is stored in the graph and mutated transactionally:

```
CYPHER 25
CREATE (:Machine {state: 0, A: 0, B: 0});

UNWIND range(1, 9223372036854775807) AS step  // 2^63 - 1
CALL (step) {
  MATCH (m:Machine)
  // 1/0 guard BEFORE list indexing
  WITH m,
       CASE WHEN m.state = -1 THEN 1/0
            ELSE $program[m.state]
       END AS instr
  SET m.state = CASE instr.op
    WHEN 'INC' THEN instr.next
    WHEN 'JZDEC' THEN CASE instr.counter
      WHEN 'A' THEN
        CASE WHEN m.A = 0
          THEN instr.q_zero ELSE instr.q_pos END
      WHEN 'B' THEN
        CASE WHEN m.B = 0
          THEN instr.q_zero ELSE instr.q_pos END
      END
    WHEN 'HALT' THEN -1 END,
  m.A = CASE
    WHEN instr.op = 'INC' AND instr.counter = 'A'
      THEN m.A + 1
    WHEN instr.op = 'JZDEC' AND instr.counter = 'A'
      AND m.A > 0 THEN m.A - 1
    ELSE m.A END,
  m.B = CASE
    WHEN instr.op = 'INC' AND instr.counter = 'B'
      THEN m.B + 1
    WHEN instr.op = 'JZDEC' AND instr.counter = 'B'
      AND m.B > 0 THEN m.B - 1
    ELSE m.B END
} IN TRANSACTIONS OF 1 ROW
  ON ERROR BREAK
```

Termination mechanism. When the machine reaches the `HALT` instruction, `state` is set to -1. On the *next* iteration, the guard `CASE WHEN m.state = -1 THEN 1/0` fires *before* any list indexing, triggering a division-by-zero error caught by `ON ERROR BREAK`, which cleanly exits the loop. This ensures `$program[-1]` is never evaluated.

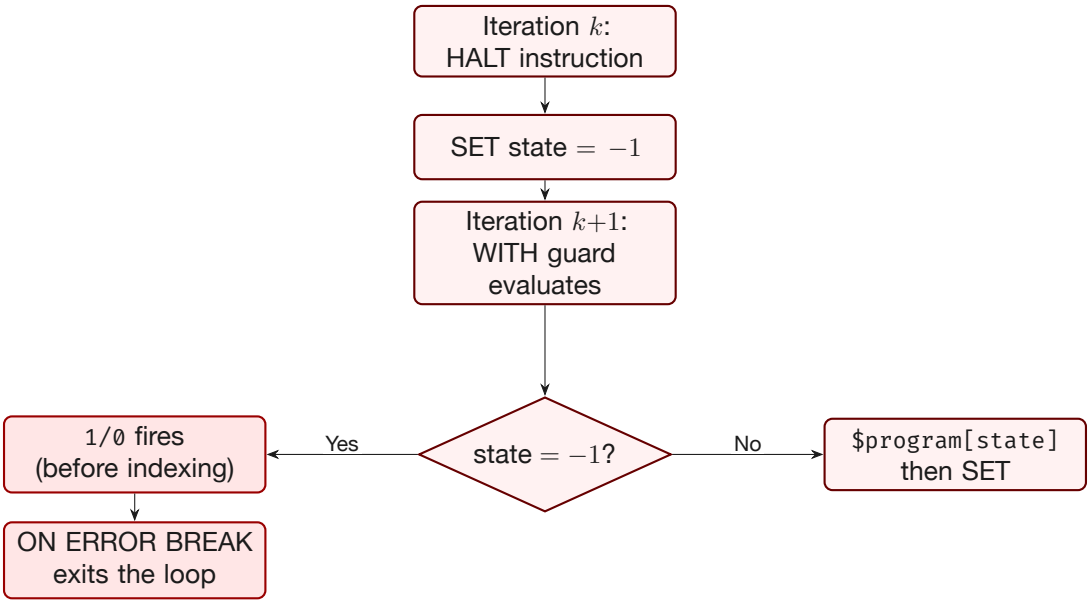

Figure 3 — Termination mechanism: the `1/0` guard fires before `$program[-1]` is ever evaluated.

Note on the 2^{63} bound. Strictly speaking, `range(1, 2^63-1)` is still a finite bound, so this does not achieve *theoretical* unbounded computation. True unbounded execution would require an external driver (e.g., a client-side loop that restarts the query if it hasn't halted). In practice, $2^{63} \approx 9.2 \times 10^{18}$ steps exceeds any physically realizable computation — the universe will end first.

§4.2 Resolution 2: Sufficiency Argument

For any Turing machine T on input w that halts in $f(|w|)$ steps:

$$\texttt{max\_steps} = f(|w|) \implies \texttt{reduce()} \text{ correctly simulates } T \text{ on } w$$

Since:

- Every **decidable** language L has a decider T_L that halts on all inputs.
- For each T_L, there exists a computable bound f.
- Cypher `reduce()` with `range(1, f(|w|))` correctly decides L.

This shows Cypher is **computationally universal for total functions**. The gap with full Turing-completeness concerns only non-halting computations (addressed by Resolution 1).

§5 Approach 2: Graph-Native QPP Traversal

Approach 1 treats Cypher as a functional language that happens to run on a graph database. This second approach embraces the graph: the machine *is* the graph, and computation *is* path finding.

§5.1 Encoding the Machine as a Graph

Each state q_i becomes a node. Each instruction becomes a typed, directed relationship. The `JZDEC` instruction is split into two relationship types — `JZDEC_ZERO` (taken when the counter is 0) and `JZDEC_POS` (taken when the counter is positive) — so that the graph captures the branching structure directly.

```
CYPHER 25
CREATE (q0:State:Init {name: 'q0'})
CREATE (q1:State      {name: 'q1'})
CREATE (q2:State      {name: 'q2'})
CREATE (q3:State:Halt {name: 'q3'})
CREATE (q0)-[:INC        {c:'A'}]->(q1)
CREATE (q1)-[:JZDEC_ZERO {c:'B'}]->(q2)
CREATE (q1)-[:JZDEC_POS  {c:'B'}]->(q3)
CREATE (q2)-[:INC        {c:'B'}]->(q0)
```

No `HALT` self-loop is needed: reaching a `:Halt` node terminates the pattern naturally.

§5.2 Simulation via `allReduce`

A valid execution trace is a path from `:Init` to `:Halt` that respects the counter constraints at every step. The `allReduce` predicate in a quantified path pattern (QPP) enforces these constraints and prunes invalid branches eagerly during traversal.

```
CYPHER 25
MATCH REPEATABLE ELEMENTS
  p = (init:Init)
    -[rels:INC|JZDEC_ZERO|JZDEC_POS]->
      {0, 9223372036854775807}
    (h:Halt)
WHERE allReduce(
  m = {A: 0, B: 0}, r IN rels |
  CASE
    WHEN r:INC AND r.c = 'A'
      THEN {A: m.A+1, B: m.B}
    WHEN r:INC AND r.c = 'B'
      THEN {A: m.A, B: m.B+1}
    WHEN r:JZDEC_POS AND r.c = 'A'
      THEN {A: m.A-1, B: m.B}
    WHEN r:JZDEC_POS AND r.c = 'B'
      THEN {A: m.A, B: m.B-1}
    ELSE m
  END,
  CASE
    WHEN r:JZDEC_ZERO AND r.c = 'A'
      THEN m.A = 0
    WHEN r:JZDEC_ZERO AND r.c = 'B'
      THEN m.B = 0
    WHEN r:JZDEC_POS AND r.c = 'A'
      THEN m.A >= 0
    WHEN r:JZDEC_POS AND r.c = 'B'
      THEN m.B >= 0
    ELSE true
  END
)
RETURN rels, length(p) AS steps
NEXT
LET final = reduce(m = {A:0, B:0},
    r IN rels |
```

```
    CASE
      WHEN r:INC AND r.c = 'A'
        THEN {A: m.A+1, B: m.B}
      WHEN r:INC AND r.c = 'B'
        THEN {A: m.A, B: m.B+1}
      WHEN r:JZDEC_POS AND r.c = 'A'
        THEN {A: m.A-1, B: m.B}
      WHEN r:JZDEC_POS AND r.c = 'B'
        THEN {A: m.A, B: m.B-1}
      ELSE m
    END
  )
RETURN steps, final.A AS ctrA, final.B AS ctrB
```

§5.3 How `allReduce` Works

`allReduce(acc, var IN groupVar | updateExpr, predicateExpr)` folds over the relationships in a QPP match, updating the accumulator at each step and checking the predicate. If the predicate ever returns `false`, the path is **pruned immediately** — the engine does not continue exploring that branch.

Subtlety: post-update predicate. The predicate is evaluated on the *post-update* accumulator. For `JZDEC_POS`, the update decrements the counter first, then the predicate checks ≥ 0 (not > 0). This ensures that decrementing from 1 to 0 is accepted.

Since `allReduce` does not expose the final accumulator value, `reduce()` in the `NEXT` stage re-derives the counter values from the surviving path (same pattern as [4]).

§5.4 Key Properties

- The machine *is* the graph; computation *is* path finding.
- `REPEATABLE ELEMENTS` allows revisiting states (loops are essential for Turing-completeness).
- `allReduce` prunes eagerly: once a `JZDEC` branch fails the predicate, that entire sub-tree is abandoned.
- The `{0, 2^63-1}` quantifier handles the degenerate case where the initial state *is* the halt state (zero-step computation).

§6 Required Primitives

§6.1 Approach 1 (`reduce()`)

Primitive	Role in the proof
`reduce()`	Fold over a list — provides **iteration**
`CASE WHEN`	**Conditional branching**
Integer $+1, -1, = 0$	**Counter operations**
Map `{state, A, B}`	Machine **state representation**
List indexing `program[i]`	$O(1)$ **instruction lookup**
`head([v IN [expr] | ...])`	Let-binding (convenience only)

No graph operations. No APOC. No GDS. Pure Cypher expressions suffice.

§6.2 Approach 2 (QPP)

Primitive	Role in the proof
`MATCH REPEATABLE ELEMENTS`	QPP with revisitable nodes
`allReduce()`	Per-step accumulator + predicate pruning
Relationship types & properties	Encode instructions
Node labels (`:Init`, `:Halt`)	Mark start and end states
`reduce()` in `NEXT`	Extract final counter values

Graph operations *are* the computation. The machine is the data model; pattern matching is the execution engine.

§7 Corollaries

Corollary 1: Undecidability of termination

Since Cypher 25 can simulate any 2-counter machine, it is *undecidable* whether a given `reduce()` expression will terminate. (By Rice's theorem applied to the reduction from 2CM.)

Corollary 2: Universality

Cypher 25 can compute any computable function. Any algorithm expressible as a Turing machine has an equivalent `reduce()`-based Cypher query.

Corollary 3: Beyond computability

While `reduce()` makes Cypher Turing-complete for *computation*, graph pattern matching with QPP and `allReduce` provides *declarative constrained traversal* with per-step pruning — something most Turing-complete languages lack. This is not about computability (all TC languages are equivalent) but about **expressiveness**: some problems are more naturally expressed as graph patterns than as imperative loops.

§8 Verified Execution on Neo4j Aura

All three constructions were tested on a Neo4j Aura instance running Neo4j 5.x with Cypher 25 support, using program P_1 from §2.

Approach 1: `reduce()` simulation

```
{A: 2, B: 0, state: -1}   ✓
```

Approach 1b: `IN TRANSACTIONS` simulation

```
m.state = -1, m.A = 2, m.B = 0   ✓
```

Approach 2: QPP `allReduce` traversal

```
steps = 5, ctrA = 2, ctrB = 0   ✓
```

All three approaches produce the expected result, matching the manual trace from §2 step-for-step.

References

[1] Minsky, M. L. (1967). *Computation: Finite and Infinite Machines*. Prentice-Hall.

[2] Neo4j Documentation: Cypher Manual.

[3] Advent of Code Cypher Solutions (2015–2025) — 300+ solved algorithmic problems. github.com/halftermeyer/adventofcode

[4] Halftermeyer, P. (2025). EV Road Trip Planning with Neo4j — QPP and `allReduce`. Neo4j Developer Blog.